AVALIAÇÃO A3 · ESTRUTURAS DE DADOS E ANÁLISE DE ALGORITMOS

Sistema de Lista de Compras

Tabela hash + lista duplamente encadeada, em Java

Marcos Paulo Madeira Ghizzo · RA 1072315250
Gabriel da Veiga Bressan · RA 1072316434
Marcio André Argenti Braz Filho · RA 1072310072
Marco Antônio Santana Costa · RA 1072312397

Prof. Hélton Ribeiro Nunes
Dezembro de 2026

O PROBLEMA

O que o sistema precisa fazer

Cadastrar

Nome, categoria, quantidade e preço estimado

Localizar

Encontrar qualquer item pelo nome

Comprar

Registrar compras totais ou parciais

Acompanhar

Pendentes, adquiridos, valor estimado e gasto

Regras do enunciado: Uma única lista | Dados só em memória | Sem vetor e sem matriz | Interface de texto

A IDEIA

A hash encontra.
A lista guarda.

Cada produto vive em um nó da lista encadeada, e a tabela hash aponta direto para esse nó.

ARQUITETURA

Três camadas, uma direção

`ui`

Interface de texto

- `ShoppingListApp`
- `ConsoleInput`
- `ConsoleView`
- `MenuOption`



`domain`

Regras de negócio

- `ShoppingList`
- `Product`
- `ShoppingSummary`
- `Money`



`datastructure`

Estruturas genéricas

- `DoublyLinkedList`
- `HashTable`
- `HashTableSnapshot`

A tela usa o domínio, e o domínio usa as estruturas. As estruturas não sabem o que é um produto: servem para qualquer tipo de dado.

ESTRUTURA 1

Lista duplamente encadeada



O(1)
Inserir no fim (pelo tail)

O(1)
Remover um nó já conhecido

O(n)
Percorrer para listar e somar

Cresce e encolhe sem realocar nada e mantém a ordem de cadastro dos produtos.

ESTRUTURA 1 · REMOÇÃO

Remover um nó em O(1)

ANTES



DEPOIS DE REMOVER FEIJÃO



Feijão fica sem ligações e sai da lista. Ninguém foi percorrido.

```
// anterior aponta para o próximo
node.previous.next = node.next;
// próximo aponta para o anterior
node.next.previous = node.previous;
```

Só é O(1) porque cada nó conhece o anterior. Numa lista simples, achar o anterior exigiria percorrer a lista: O(n).

Nas pontas: sem anterior, o head passa a ser o próximo; sem próximo, o tail passa a ser o anterior.

ESTRUTURA 2

Tabela hash com encadeamento

“Feijão Preto 1kg”

↓ tira acentos, maiúsculas e espaços extras

“feijao preto 1kg”

↓ hashCode() mod 16

bucket 9

16 BUCKETS · CADA UM É UMA LISTA ENCADEADA

[0]	leite integral 1l	
[1]	detergente → oleo de soja 900ml	colisão
[2]	macarrao 500g	
[9]	feijao preto 1kg	
[13]	arroz 5kg	

... os buckets que não aparecem estão vazios

Por isso “FEIJAO PRETO 1KG” e “feijão preto 1kg” encontram o mesmo produto.

ESTRUTURA 2 · FATOR DE CARGA

Quando a tabela enche, ela dobra

0,75

fator de carga máximo

Fator de carga = chaves ÷ buckets. Passou de 0,75, a tabela dobra de tamanho e redistribui todas as chaves: é o rehash.

Assim as listas dos buckets ficam curtas e as operações continuam O(1) em média.

AO CADASTRAR O 13º PRODUTO

16 buckets · 13 chaves

Fator 0,81: passou do limite



DEPOIS DO REHASH

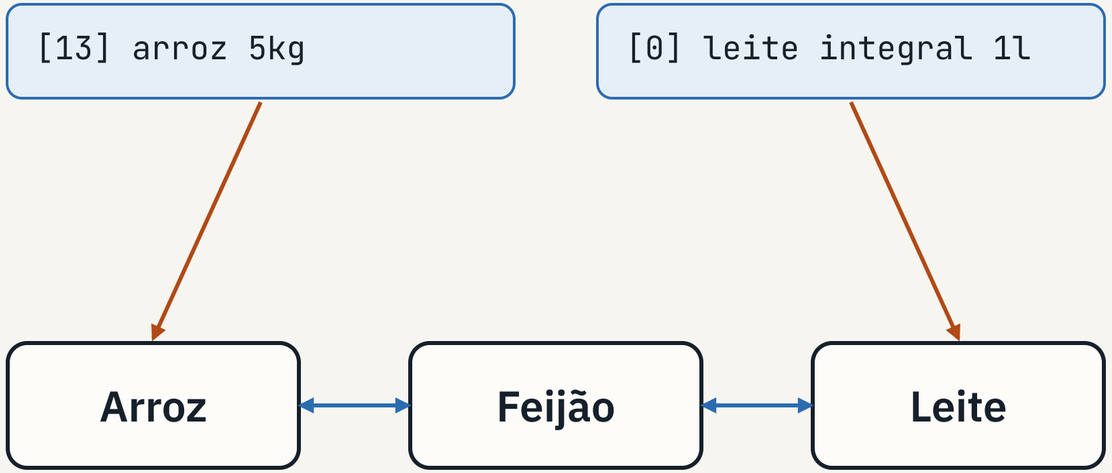
32 buckets · 13 chaves

Fator 0,41: listas curtas de novo

AS DUAS JUNTAS

A hash guarda o nó, não o produto

TABELA HASH (ÍNDICE)



LISTA ENCADEADA (DADOS)

REMOVER “ARROZ 5KG”

1. A hash devolve o nó do produto: O(1)
2. A lista religa os vizinhos do nó: O(1)
3. A hash apaga a chave: O(1)

Remoção completa em O(1) médio, sem percorrer nada.

ANÁLISE DE COMPLEXIDADE

Quanto custa cada operação

Operação	Como é feita	Custo
Cadastrar	hash evita duplicado; lista insere no fim	**O(1)***
Consultar	hash busca pelo nome normalizado	**O(1)***
Comprar ou editar	localiza na hash e atualiza o produto	**O(1)***
Remover	hash entrega o nó; lista e hash removem	**O(1)***
Listar	percorre a lista e filtra	**O(n)**
Resumo de valores	percorre a lista uma vez somando	**O(n)**

* Em média. Pior caso O(n), se todas as chaves caírem no mesmo bucket; o rehash evita isso na prática.

REGRA DE NEGÓCIO

Compra total e parcial: leite 1L

PASSO 1

Cadastro

12 unidades a R$ 5,29 cada (estimado)

0/12

Pendente

Gasto: R$ 0,00

PASSO 2

Compra parcial

6 unidades a R$ 5,49 (preço pago)

6/12

Parcial

Gasto: R$ 32,94

PASSO 3

Compra do resto

Enter compra as 6 que faltam a R$ 5,29

12/12

Comprado

Gasto: R$ 64,68

Pendente, status e valores são calculados a partir dos dados, nunca digitados.

CONSISTÊNCIA

Dados e cálculos sempre certos

Dinheiro com BigDecimal

Valores com 2 casas. Com double, 0,1 + 0,2 daria 0,30000000000000004.

Nada derivado é guardado

Pendente, status e totais são calculados na hora, então não ficam desatualizados.

Uma porta de entrada

Só a ShoppingList altera produtos, e mantém hash e lista sincronizadas.

Validação no domínio

Sem preço negativo, nome repetido ou compra acima do que falta.

QUALIDADE DO CÓDIGO

Clean Code na prática

Nomes que explicam

pendingQuantity(), requireNameAvailable(), bypassFromPrevious()

Funções pequenas

Cada opção do menu é um método curto, com um único objetivo.

Uma responsabilidade por classe

Estrutura, regra, leitura do teclado e tela ficam separadas.

Sem retornar null

Buscas devolvem Optional: o “não encontrado” fica explícito.

Erros como exceções

Mensagens claras, tratadas num único ponto da interface.

Sem números mágicos

MAX_LOAD_FACTOR, GROWTH_FACTOR e DEFAULT_CAPACITY têm nome.

Javadoc só para o porquê e a complexidade; o que o código faz, os nomes já dizem.

TESTES

Cada regra tem um teste

34

testes automatizados, 0 falhas

Rodam com um comando e não usam bibliotecas externas: o executor também foi feito por nós.

```
.\test.bat
```

Lista encadeada ordem, remoção no início, no meio e no fim, nó de outra lista	6
Tabela hash colisões forçadas, rehash e 1.000 chaves	6
Produto e nome normalizado compra parcial, limites, validações e acentos	8
Lista de compras nomes duplicados, remoção e resumo consistente	6
Interface valores em R$ e um usuário simulado digitando no menu	8

AO VIVO

Demonstração

- **10** Carregar a lista de exemplo e listar tudo (4)
- **2** Buscar “feijao preto 1kg”, digitado sem acento
- **3** Comprar 4 leites a R$ 5,10; depois o resto com Enter
- **5 6** Itens pendentes e adquiridos, com os totais
- **7** Resumo: gasto × estimado e previsão final
- **11** Ver a colisão; cadastrar 3 produtos e ver o rehash
- **9** Remover um produto e conferir a lista (4) e a hash (11)
- **1** Erro tratado: cadastrar “ARROZ 5KG” de novo

OBRIGADO

Perguntas?

A hash encontra, a lista guarda, e a ShoppingList mantém as duas sincronizadas.

Marcos Paulo Madeira Ghizzo · RA 1072315250
Gabriel da Veiga Bressan · RA 1072316434
Marcio André Argenti Braz Filho · RA 1072310072
Marco Antônio Santana Costa · RA 1072312397

```
.\run.bat
```

```
.\test.bat
```